

1 Domande in preparazione all'esame di Ingegneria Del Software

1.1 Modelli di processo e di sviluppo software

- 1) Che cos'è un processo di sviluppo software? → pag 9
- 2) Modello Code & Fix → pag 11
- 3) Modello a cascata (Waterfall) e le sue 2 varianti → pag 11
- 4) Che cos'è uno studio di fattibilità? → pag 12
- 5) Come funziona il V-MODEL? → pag 13
- 6) Modelli evolutivi - Prototipazione → pag 13
- 7) Modello Interattivo-Incrementale → pag 14
- 8) Modello a spirale - Che cos'è un rischio? → pag 14
- 9) Unified Process(UP) → pag 15
- 10) Component-Based Software Engineering (CBSE) → pag 16
- 11) Modello trasformazionale e metodi formali → pag 16
- 12) Approcci plan-driven e metodi agili → pag 17
- 13) Come scegliere un modello? → pag 19

1.2 Ingegneria dei requisiti

- 1) Che cos'è un requisito? → pag 20
- 2) Requisiti utente e di sistema → pag 21
- 3) Requisiti funzionali e non funzionali → pag 23

1.3 Use case

- 1) Che cos'è uno use case? - Da cosa è composto? → pag 25
- 2) Quali sono le limitazioni nella creazione di uno use case? → pag 26
- 3) Che cos'è uno scenario? → pag 27
- 4) Derivazioni, ripetizioni, scenari alternativi → pag 29
- 5) Screen mockup → pag 32
- 6) Punti di forza e di debolezza → pag 32

1.4 Design architetturale

- 1) Che cos'è il design e da quali livelli è composto? (livelli di raffinamento e di riuso) → pag 33
- 2) Da cosa è composta l'architettura software? → pag 33
- 3) Layered architecture → pag 35
- 4) Repository Model → pag 35
- 5) Client-Server Model → pag 36
- 6) Two-tier e three-tier → pag 36
- 7) Peer-to-peer (P2P) → pag 36
- 8) Pipe and filter → pag 37
- 9) Stili di controllo : Call-return, Manager Model, Broadcast → pag 38
- 10) Esempio di architettura eterogenea → pag 40
- 11) Microservizi → pag 41

1.5 Design delle componenti

- 1) Che cos'è il lowlevel design e quali sono le sue componenti? → pag 43
- 2) Che cos'è il design by contract? → pag 43
- 3) Quali sono le responsabilità del cliente e quali del fornitore? → pag 44
- 4) Che tipi di notazioni vengono fatte durante la progettazione di algoritmi?
→ pag 46
- 5) Notazione e PDL - Stepwise refinement → pag 46
- 6) Modulo - Astrazione - Decomposizione → pag 47
- 7) Che cos'è la modularità e Separation of Concerns? → pag 47
- 8) Information hiding → pag 48

1.6 Introduzione a UML

- 1) Che cos'è UML e cosa UML non è? → pag 50
- 2) Sketch, blueprint e linguaggio eseguibile → pag 50
- 3) Prospettiva concettuale e prospettiva software → pag 50
- 4) Modello di dominio → pag 51
- 5) Notazione, meta-modello e viste → pag 52
- 6) Modello 4+1 → pag 52
- 7) Diagrammi di UML 2.5 e diagrammi principali nel corso → pag 53

- 8) **UML nel processo di sviluppo** → pag 54
- 9) **Profili UML, codice e uso pragmatico** → pag 55

1.7 Class diagram e Object diagram UML

- 1) **Che cos'è un Class Diagram? Classi, feature e relazioni** → pag 57
- 2) **Notazione della classe: nome, attributi e operazioni** → pag 57
- 3) **Attributi: sintassi, visibilità e molteplicità** → pag 57
- 4) **Operazioni, datatype, costruttori e membri statici** → pag 58
- 5) **Associazioni e molteplicità: nomi, ruoli e navigabilità** → pag 59
- 6) **Attributo o associazione? Associazioni riflessive e bidirezionali** → pag 59
- 7) **Object Diagram** → pag 60
- 8) **Classe associativa** → pag 60
- 9) **Aggregazione e composizione** → pag 61
- 10) **Generalizzazione ed ereditarietà** → pag 61
- 11) **Dipendenze e traduzione nel codice** → pag 62
- 12) **Dalle molteplicità alle strutture dati** → pag 62

1.8 Interfacce e vincoli UML

- 1) **Interfacce fornite e richieste** → pag 64
- 2) **Realizzazione di un'interfaccia e interfaccia richiesta** → pag 64
- 3) **Due notazioni equivalenti per le interfacce** → pag 65
- 4) **Perché usare le interfacce?** → pag 66
- 5) **Vincoli nei Class Diagram** → pag 67
- 6) **Object Constraint Language (OCL)** → pag 67

1.9 State Machine Diagram

- 1) **Che cos'è uno State Machine Diagram? Stato e comportamento** → pag 68
- 2) **Modellazione statica e dinamica** → pag 68
- 3) **Stati, transizioni e pseudo-stati** → pag 69
- 4) **Eventi e run-to-completion** → pag 69
- 5) **Esempio: ciclo di vita di una copia di libro** → pag 70
- 6) **Attività interne: entry, exit, do e reazioni interne** → pag 71

- 7) Stati composti e regioni ortogonali → pag 72
- 8) Quando usare una State Machine? Forno e workflow documentale → pag 73
- 9) Implementazione: switch, Design Pattern State e tabelle di stato → pag 74

1.10 Sequence e Communication Diagram

- 1) Fondamenti dei Sequence Diagram: partecipanti, lifeline e attivazioni → pag 75
- 2) Messaggi sincroni e asincroni → pag 75
- 3) Dal diagramma al codice → pag 76
- 4) Ciclo di vita dei partecipanti → pag 77
- 5) Frame di interazione: loop, opt e alt → pag 78
- 6) Frame di interazione: par, ref e sd → pag 79
- 7) Sequence Diagram di sistema → pag 80
- 8) Sequence Diagram nel design: esempio del registro di cassa → pag 81
- 9) Esempio prezzo di un ordine: soluzione centralizzata → pag 82
- 10) Pseudocodice della soluzione centralizzata → pag 83
- 11) Esempio prezzo di un ordine: soluzione distribuita → pag 84
- 12) Pseudocodice della soluzione distribuita → pag 85
- 13) Communication Diagram e linee guida → pag 86

1.11 Activity Diagram

- 1) Concetti fondamentali: azioni, nodi e flussi → pag 87
- 2) Semantica dei token → pag 87
- 3) Esempio: gestione di un ordine ed evoluzione dei token → pag 88
- 4) Nodi di controllo: inizio, terminazione, decisione, fusione, fork e join → pag 89
- 5) Nodo finale di flusso → pag 90
- 6) Oggetti, pin e stati → pag 91
- 7) Partizioni e responsabilità → pag 92
- 8) Eventi e interruzioni: invio, ricezione e regioni interrompibili → pag 93
- 9) Parametri e valori di ritorno: fare il bucato → pag 95
- 10) Modellare Use Case e operazioni: ATM e authenticateUser → pag 96
- 11) Modellare un algoritmo: ricerca binaria → pag 98

1.12 Component, Deployment e Package Diagram

- 1) **Component Diagram: concetto di componente** → pag 99
- 2) **Componenti e artefatti** → pag 99
- 3) **Perché componenti e non classi? Notazione dei componenti** → pag 100
- 4) **Interfacce fornite/richieste, porte e vista interna** → pag 100
- 5) **Dipendenze tra componenti** → pag 101
- 6) **Esempio: sistema di gestione ordini** → pag 102
- 7) **Deployment Diagram: nodi, connessioni, artefatti e manifest** → pag 103
- 8) **Nodi annidati e tagged value** → pag 104
- 9) **Esempio officina meccanica: requisiti, architettura hardware e software** → pag 105
- 10) **Integrazione Component e Deployment** → pag 106
- 11) **Package Diagram: package, namespace, visibilità e dipendenze** → pag 107
- 12) **Progettare i package: cicli e dipendenze entranti** → pag 108
- 13) **Architettura A/B, procedura pratica e risoluzione dei cicli** → pag 109
- 14) **Package: non soltanto classi** → pag 110

1.13 Software Design Pattern

- 1) **Che cos'è un Software Design Pattern e perché usarlo?** → pag 111
- 2) **Panorama del riuso: librerie, framework, componenti e pattern** → pag 111
- 3) **GRASP e pattern classici** → pag 112
- 4) **Come si descrive un pattern?** → pag 113
- 5) **Abstract Factory: dalla creazione alla fabbrica e quando usarla** → pag 114
- 6) **Abstract Factory: esempio GUI multiplatforma e conseguenze** → pag 114
- 7) **Abstract Factory: struttura d'implementazione** → pag 115
- 8) **Adapter: problema, scopo ed esempio del cerchio** → pag 116
- 9) **Object Adapter e Class Adapter** → pag 116
- 10) **Facade: ingresso semplice a un sottosistema** → pag 117
- 11) **Facade: esempi, conseguenze e confronto con Adapter** → pag 117
- 12) **Template Method: scheletro, passi variabili e login riutilizzabile** → pag 118
- 13) **Template Method: inversione di controllo e conseguenze** → pag 118
- 14) **Observer: problema, registrazione e notifica** → pag 119
- 15) **Observer: conseguenze** → pag 120

- 16) **State: comportamento dipendente dallo stato e distributore automatico** → pag 121
- 17) **State: implementazione con il pattern** → pag 121
- 18) **State: struttura generale e conseguenze** → pag 122

1.14 Software Testing

- 1) **Terminologia attraverso il caso InvestireOK** → pag 124
- 2) **Recovery, fault localization, fault removal e testing di regressione** → pag 124
- 3) **Errore, fault e failure** → pag 125
- 4) **Esempio reDouble e fault latente** → pag 125
- 5) **Testing e debugging: SUT, oracolo e differenze** → pag 126
- 6) **Perché il testing è difficile? Testing esaustivo** → pag 127
- 7) **White-box e black-box** → pag 127
- 8) **Test case e testsuite** → pag 128
- 9) **Livelli di testing: unità, integrazione, sistema, accettazione e regressione** → pag 129
- 10) **Automazione e JUnit 5** → pag 130
- 11) **Perché non usare main per i test?** → pag 130
- 12) **Primo caso di test JUnit: classe, metodo e asserzioni** → pag 131
- 13) **Esecuzione verde/rosso, failure ed error nel test runner** → pag 131
- 14) **Progettare test coesi: una cosa alla volta** → pag 132
- 15) **Fixture: BeforeEach e AfterEach** → pag 132
- 16) **Ciclo di vita completo: BeforeAll e AfterAll** → pag 133
- 17) **Asserzioni JUnit: esecuzione, arresto e operazioni principali** → pag 134
- 18) **Test parametrici** → pag 135